

PAINTING BUSINESS

CORE INSTANCE

Documentation technique — Module Auth + Acteurs

Spring Boot 3 • Java 17 • PostgreSQL • JWT • Bean Validation

Membre	Rôle	Module
Abel	Auth + Acteurs	RegisterDto • LoginDto • AuthResponseDto AuthController • ConnecterService InscrireService • ActorController ValiderCompteService • Tests

TABLE DES MATIÈRES

1	Architecture générale du projet	3
2	SEMAINE 1 — Les DTOs et Bean Validation	4
2.1	Qu'est-ce qu'un DTO ?	4
2.2	RegisterDto — Inscription	5
2.3	LoginDto — Connexion	6
2.4	AuthResponseDto — Réponse JWT	6
3	L'entité Utilisateur et l'Enum des rôles	7
4	La sécurité : JWT expliqué simplement	8
4.1	JwtService — Créer et valider les tokens	9
4.2	JwtAuthFilter — Interceptor les requêtes	10
4.3	SecurityConfig — Configurer Spring Security	11
5	SEMAINE 2 — Les Services métier	12
5.1	InscrireService	12
5.2	ConnecterService	13
6	SEMAINE 2 — Les Contrôleurs REST	14
6.1	AuthController — Endpoints d'auth	14
6.2	ActorController — Gestion des acteurs	15
7	SEMAINE 3 — ValiderCompteService	16
8	SEMAINE 3 — Les Tests	17
8.1	Tests de AuthController	17
8.2	Tests de InscrireService	18
8.3	Tests de ActorController	18
9	Configuration — pom.xml & application.properties	19
10	Résumé des endpoints API	20

1. Architecture générale du projet

Le projet **Painting Business Core Instance** est une application web construite en couches. Chaque couche a une responsabilité précise et ne communique qu'avec la couche adjacente. Cette séparation rend le code maintenable, testable et évolutif.

Couche	Rôle	Exemples dans notre projet
Presentation (Controller)	Reçoit les requêtes HTTP et retourne une réponse	<code>AuthController</code> , <code>ActorController</code>
Service (Business Logic)	Contient toute la logique métier	<code>InscrireService</code> , <code>ConnecterService</code> , <code>ValiderCompteService</code>
Repository (Data Access)	Communique avec la base de données	<code>UtilisateurRepository</code> (fourni par Cesare)
Entity (Domain Model)	Représente les tables de la base de données	<code>Utilisateur</code> , <code>RoleUtilisateur</code>
DTO (Transfer Object)	Transporte les données entre couches	<code>RegisterDto</code> , <code>LoginDto</code> , <code>AuthResponseDto</code>
Security	Gère l'authentification et l'autorisation	<code>JwtService</code> , <code>JwtAuthFilter</code> , <code>SecurityConfig</code>

Flux d'une requête d'inscription (exemple concret)

1. Le front-end envoie un POST `/api/auth/inscription` avec les données JSON
2. **JwtAuthFilter** vérifie — pas de token requis pour cet endpoint (public)
3. **AuthController** reçoit la requête et valide le DTO avec Bean Validation
4. **InscrireService** vérifie l'email, hash le mot de passe, sauvegarde en BDD
5. **JwtService** génère un token JWT
6. **AuthResponseDto** est retourné au client avec le token JWT et les infos

2. SEMAINE 1 — Les DTOs et Bean Validation

2.1 — Qu'est-ce qu'un DTO ?

Un **DTO (Data Transfer Object)** est une classe Java simple dont le seul rôle est de transporter des données. Il ne contient pas de logique métier, juste des attributs, des constructeurs et des getters/setters.

Pourquoi ne pas utiliser directement l'entité *Utilisateur* ? **Parce que l'entité contient des informations sensibles** (mot de passe hashé, token de validation...) que l'on ne doit jamais exposer à l'extérieur. Le DTO expose uniquement ce qui est nécessaire.

Entité Utilisateur (base de données)	RegisterDto (ce que le client envoie)
id, prenom, nom, email, motDePasse (hashé), role, compteValide, compteActif, dateCreation, tokenValidation	prenom, nom, email, motDePasse (clair), confirmationMotDePasse, role

2.2 — Bean Validation : valider les données automatiquement

Bean Validation est un standard Java qui permet de définir des règles de validation directement sur les attributs d'une classe avec des annotations. Spring Boot les vérifie automatiquement grâce à **@Valid** dans le contrôleur.

Annotation	Ce qu'elle vérifie	Exemple
@NotBlank	Champ non vide et non null	@NotBlank(message="Obligatoire")
@Email	Format email valide	@Email(message="Email invalide")
@Size	Longueur min/max	@Size(min=8, max=50)
@Pattern	Correspond à une regex	@Pattern(regexp="^(?=.*[A-Z]).*\$")
@NotNull	Valeur non null	@NotNull
@Min/@Max	Valeur numérique min/max	@Min(1) @Max(100)

2.3 — RegisterDto : le formulaire d'inscription

Ce DTO représente exactement ce qu'un utilisateur remplit quand il s'inscrit sur l'application de peinture. Il contient des contraintes strictes pour garantir la qualité des données avant même d'arriver en base de données.

```
# Extraits clés de RegisterDto.java
@NotBlank(message = "Le prénom est obligatoire")
@Size(min = 2, max = 50, message = "Entre 2 et 50 caractères")
private String prenom;

@NotBlank(message = "L'email est obligatoire")
@email(message = "L'adresse email n'est pas valide")
private String email;

@NotBlank
@Size(min = 8, message = "Au moins 8 caractères")
@Pattern(
    regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).*$",
    message = "Doit contenir majuscule, minuscule et chiffre"
```

```
)  
private String motDePasse;  
  
@Pattern(regexp = "PEINTRE|CLIENT|GALERIE")  
private String role;
```

2.4 — AuthResponseDto : ce que le serveur renvoie

Après une connexion ou inscription réussie, le serveur ne renvoie jamais le mot de passe. Il retourne ce DTO qui contient uniquement le **token JWT** et les informations utiles au front-end (nom, rôle, statut du compte).

```
# Exemple de réponse JSON retournée par l'API  
// Ce que reçoit le client après s'être connecté :  
{  
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqZWFuQHBhaw50...",  
  "type": "Bearer",  
  "utilisateurId": 42,  
  "email": "jean@painting.fr",  
  "nomComplet": "Jean Dupont",  
  "role": "PEINTRE",  
  "compteValide": true  
}
```

3. L'entité Utilisateur et les rôles

L'entité **Utilisateur** est la représentation Java de la table *utilisateurs* en base de données PostgreSQL. Les annotations JPA (Jakarta Persistence API) indiquent à Hibernate comment mapper cette classe sur la base.

Annotation	Rôle
@Entity	Déclare que cette classe est une table en BDD
@Table(name=...)	Spécifie le nom exact de la table SQL
@Id	Marque la clé primaire
@GeneratedValue	L'ID est auto-incrémenté par PostgreSQL
@Column	Configure la colonne (nullable, unique, length)
@Enumerated	Stocke l'enum comme String (ex: "PEINTRE")
@PrePersist	Méthode appelée automatiquement avant l'insertion

Les 3 rôles de l'application de peinture — définis par l'enum **RoleUtilisateur** :

Rôle	Description
PEINTRE	Artiste qui publie des fiches d'oeuvres et des roadmaps
CLIENT	Visiteur/acheteur qui consulte et recherche des oeuvres
GALERIE	Gestionnaire d'espace d'exposition
ADMIN	Administrateur de la plateforme (accès total)

4. La sécurité : JWT expliqué simplement

Un **JWT (JSON Web Token)** est comme un badge d'entrée numérique. Quand tu t'inscris ou te connectes, le serveur te donne ce badge. À chaque requête suivante, tu présentes ce badge, et le serveur sait qui tu es sans avoir à retourner en base de données.

```
# Anatomie d'un JWT
# Structure d'un JWT (3 parties séparées par des points)
eyJhbGciOiJIUzI1NiJ9      <- HEADER (algorithme utilisé)
.eyJzdWUiOiJqZWFuQHBhaw50 <- PAYLOAD (données : email, rôle, expiration)
.SflKxwRJSMeKKF2QT4fwpMeJf <- SIGNATURE (garantit que personne n'a modifié le token)
```

Pourquoi JWT et pas les sessions HTTP traditionnelles ?

Sessions HTTP : Le serveur mémorise chaque utilisateur connecté (état). Problème : si on a plusieurs serveurs (scalabilité), ils ne partagent pas la même mémoire.

JWT : Le token contient toutes les infos nécessaires, signé cryptographiquement. Le serveur n'a pas besoin de mémoriser quoi que ce soit (STATELESS). N'importe quel serveur peut vérifier le token en recalculant la signature.

4.1 — JwtService : créer et valider les tokens

```
# JwtService.java - Génération d'un token
// Générer un token pour un utilisateur
public String genererToken(String email, String role) {
    Key key = Keys.hmacShaKeyFor(secretKey.getBytes());

    return Jwts.builder()
        .setSubject(email)           // identifiant principal
        .claim("role", role)         // données personnalisées
        .setIssuedAt(new Date())     // date de création
        .setExpiration(              // date d'expiration (24h)
            new Date(System.currentTimeMillis() + expirationMs))
        .signWith(key, SignatureAlgorithm.HS256)
        .compact();
}
```

4.2 — JwtAuthFilter : le filtre qui vérifie chaque requête

Ce filtre s'exécute **avant chaque requête HTTP**. Il lit l'en-tête *Authorization: Bearer <token>*, valide le token et identifie l'utilisateur.

```
# JwtAuthFilter.java - Logique principale
// Extrait de JwtAuthFilter.java
String authHeader = request.getHeader("Authorization");

// Vérifie que l'en-tête commence par "Bearer "
if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response); // Endpoint public
    return;
}

String token = authHeader.substring(7); // Enlève "Bearer "
```

```
if (jwtService.estValide(token)) {  
    String email = jwtService.extraireEmail(token);  
    String role = jwtService.extraireRole(token);  
    // Enregistre l'utilisateur dans le contexte Spring Security  
    SecurityContextHolder.getContext().setAuthentication(...);  
}
```


5. SEMAINE 2 — Les Services métier

Les services contiennent toute la **logique métier**. Ils ne savent pas comment les données arrivent (HTTP, WebSocket...) ni comment elles sont stockées (PostgreSQL, fichier...). Cette séparation est fondamentale.

5.1 — InscrireService : les étapes de l'inscription

Étape 1	Vérifier que l'email n'est pas déjà utilisé → sinon 400
Étape 2	Vérifier que les mots de passe correspondent → sinon 400
Étape 3	Hasher le mot de passe avec BCrypt (jamais stocker en clair !)
Étape 4	Créer l'entité Utilisateur et générer un UUID de validation
Étape 5	Sauvegarder en base de données via UtilisateurRepository.save()
Étape 6	(S3) Envoyer l'email de validation avec le token UUID
Étape 7	Générer un token JWT et retourner AuthResponseDto

```
# InscrireService.java
// Extrait clé de InscrireService.java
public AuthResponseDto inscrire(RegisterDto dto) {

    // Règle 1 : email unique
    if (utilisateurRepository.existsByEmail(dto.getEmail())) {
        throw new IllegalArgumentException("Email déjà utilisé");
    }

    // Règle 2 : mots de passe identiques
    if (!dto.getMotDePasse().equals(dto.getConfirmationMotDePasse())) {
        throw new IllegalArgumentException("Mots de passe différents");
    }

    // Hash du mot de passe (JAMAIS stocker en clair)
    String motDePasseHashe = passwordEncoder.encode(dto.getMotDePasse());

    // Création et sauvegarde
    Utilisateur u = new Utilisateur(dto.getPrenom(), dto.getNom(),
        dto.getEmail(), motDePasseHashe, RoleUtilisateur.valueOf(dto.getRole()));
    u.setTokenValidation(UUID.randomUUID().toString());
    Utilisateur sauvegarde = utilisateurRepository.save(u);

    // Génération JWT
    String token = jwtService.genererToken(sauvegarde.getEmail(), ...);
    return new AuthResponseDto(token, ...);
}
```

5.2 — ConnecterService : vérifier l'identité

```
# ConnecterService.java – Vérification du mot de passe
// Le BCrypt ne "décrypte" pas : il re-hashes et compare
// passwordEncoder.matches("MonMdpClair", "$2b$12$HASH...") -> true/false

Utilisateur u = utilisateurRepository.findByEmail(dto.getEmail())
    .orElseThrow(() -> new IllegalArgumentException(
        "Email ou mot de passe incorrect" // Message vague = sécurité
    ));
```

```
if (!passwordEncoder.matches(dto.getMotDePasse(), u.getMotDePasse())) {  
    throw new IllegalArgumentException("Email ou mot de passe incorrect");  
}
```

6. SEMAINE 2 — Les Contrôleurs REST

Un contrôleur REST est la **porte d'entrée HTTP** de l'application. Il reçoit les requêtes, délègue au service, et retourne la bonne réponse HTTP.

Code HTTP	Signification	Quand l'utiliser
200 OK	Succès	Connexion réussie, données récupérées
201 Created	Ressource créée	Inscription réussie
400 Bad Request	Données invalides	Email déjà pris, validation échouée
401 Unauthorized	Non authentifié	Token manquant ou invalide
403 Forbidden	Accès refusé	Rôle insuffisant (PEINTRE vs ADMIN)
404 Not Found	Ressource introuvable	Utilisateur inexistant

6.1 — AuthController : les 4 endpoints d'authentification

```
# AuthController.java — Endpoint d'inscription
// @Valid déclenche Bean Validation automatiquement
// Si une contrainte est violée → 400 Bad Request automatique

@PostMapping("/inscription")
public ResponseEntity inscription(@Valid @RequestBody RegisterDto dto) {
    try {
        AuthResponseDto reponse = inscrireService.inscrire(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(reponse); // 201
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest() // 400
            .body(Map.of("erreur", e.getMessage()));
    }
}
```

6.2 — ActorController : la gestion des acteurs par rôle

L'annotation **@PreAuthorize** vérifie le rôle de l'utilisateur *avant* d'exécuter la méthode. Si le rôle est insuffisant, Spring Security retourne automatiquement 403 Forbidden.

```
# ActorController.java — Contrôle par rôle
// Accessible à tous les utilisateurs connectés
@GetMapping("/moi")
public ResponseEntity monProfil(Authentication auth) { ... }

// Accessible uniquement aux ADMIN
@GetMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity listerTousLesActeurs() { ... }

// Désactiver un compte — ADMIN seulement
@PatchMapping("/{id}/desactiver")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity desactiverCompte(@PathVariable Long id) { ... }
```

7. SEMAINE 3 — ValiderCompteService

La validation de compte par email est une étape essentielle pour garantir que l'utilisateur possède bien l'adresse email fournie. Sans cette validation, n'importe qui pourrait s'inscrire avec l'email de quelqu'un d'autre.

- | | |
|---|--|
| 1 | Utilisateur s'inscrit → tokenValidation = UUID aléatoire sauvegardé en BDD |
| 2 | (TODO S3) Email envoyé : https://painting.fr/api/auth/valider?token=UUID |
| 3 | Utilisateur clique sur le lien → GET /api/auth/valider?token=UUID |
| 4 | ValiderCompteService cherche l'utilisateur par ce token |
| 5 | compteValide = true, dateValidation = maintenant, tokenValidation = null |
| 6 | L'utilisateur peut maintenant accéder à toutes les fonctionnalités |

```
# ValiderCompteService.java

public void validerCompte(String token) {
    // Cherche l'utilisateur par son token UUID
    Utilisateur u = utilisateurRepository
        .findByTokenValidation(token)
        .orElseThrow(() -> new IllegalArgumentException("Token invalide"));

    if (u.isCompteValide()) {
        throw new IllegalArgumentException("Compte déjà validé");
    }

    // Validation
    u.setCompteValide(true);
    u.setDateValidation(LocalDate.now());
    u.setTokenValidation(null); // Efface le token après usage
    utilisateurRepository.save(u);
}
```

8. SEMAINE 3 — Les Tests

Les tests permettent de s'assurer que le code fonctionne correctement et qu'une modification future ne casse pas ce qui marchait avant. Dans Spring Boot, on utilise **JUnit 5** + **Mockito**.

Outil	Rôle
@WebMvcTest	Charge uniquement la couche HTTP (pas la BDD)
@ExtendWith(Mockito)	Active les mocks dans un test de service
@MockBean	Remplace un bean Spring par un mock simulé
@Mock / @InjectMocks	Crée des mocks Mockito et injecte dans le service testé
when(...).thenReturn()	Définit ce que retourne un mock quand on l'appelle
@WithMockUser	Simule un utilisateur connecté avec un rôle précis
assertThat(...)	Vérifie une valeur (bibliothèque AssertJ)
verify(...)	Vérifie qu'une méthode a été (ou pas) appelée

8.1 — Anatomie d'un test (pattern AAA)

```
# AuthControllerTest.java — Pattern AAA
// Pattern AAA : Arrange / Act / Assert

@Test
@DisplayName("Inscription valide → 201 Created avec token JWT")
void inscription_succes_retourne_201() throws Exception {

    // ARRANGE (préparer les données)
    RegisterDto dto = new RegisterDto("Jean", "Dupont",
        "jean@painting.fr", "MotDePasse1!", "MotDePasse1!", "PEINTRE");
    when(inscrireService.inscrire(any())).thenReturn(reponseSimulee);

    // ACT (exécuter l'action)
    mockMvc.perform(
        post("/api/auth/inscription")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(dto))
    )
    // ASSERT (vérifier le résultat)
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.token").value("token.jwt.fake"));
}
```

8.2 — Tester les cas d'erreur

Tester les erreurs est aussi important que tester le succès. On vérifie que le code réagit correctement aux situations anormales.

```
# Tester les cas d'erreur
// Test : email déjà utilisé → 400
when(inscrireService.inscrire(any()))
    .thenThrow(new IllegalArgumentException("Email déjà utilisé"));

mockMvc.perform(post("/api/auth/inscription")...)
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.erreur").exists());
```

```
// Test : vérifier qu'on n'a JAMAIS sauvegardé si email existant  
verify(utilisateurRepository, never()).save(any());
```

9. Configuration — pom.xml & application.properties

Dépendances clés dans le pom.xml :

spring-boot-starter-web	Crée les contrôleurs REST et le serveur HTTP (Tomcat)
spring-boot-starter-security	Ajoute Spring Security (authentification/autorisation)
spring-boot-starter-data-jpa	Active JPA/Hibernate pour la persistance BDD
spring-boot-starter-validation	Active Bean Validation (@NotBlank, @Email, etc.)
postgresql	Driver JDBC pour communiquer avec PostgreSQL
jjwt-api/impl/jackson	Librairie pour créer et valider les tokens JWT
spring-boot-starter-test	JUnit 5, Mockito, MockMvc pour les tests
spring-security-test	@WithMockUser pour simuler un utilisateur dans les tests
h2	Base de données en mémoire pour les tests (scope test)

Points importants dans application.properties

- **spring.jpa.hibernate.ddl-auto=validate** — En production, Liquibase (Cesare) crée les tables. Hibernate ne fait que vérifier qu'elles existent.
- **app.jwt.secret** — Changer cette clé en production ! Elle doit être longue (min. 256 bits) et secrète. Ne jamais la committer dans Git.
- **app.jwt.expiration-ms=86400000** — Le token expire après 24 heures (86400000 ms).

10. Récapitulatif des endpoints API

Méthode	URL	Auth requis	Rôle minimum	Description
POST	/api/auth/inscription	Non	—	Créer un compte
POST	/api/auth/connexion	Non	—	Se connecter
GET	/api/auth/valider?token=...	Non	—	Valider le compte
POST	/api/auth/renvoyer-validation	Non	—	Renvoyer l'email de validation
GET	/api/acteurs/moi	Oui	Tous	Voir son profil
GET	/api/acteurs/peintres	Oui	Tous	Liste des peintres validés
GET	/api/acteurs	Oui	ADMIN	Tous les utilisateurs
PATCH	/api/acteurs/{id}/desactiver	Oui	ADMIN	Désactiver un compte

Structure des fichiers produits

```
src/main/java/com/painting/auth/  
  dto/      → RegisterDto.java, LoginDto.java, AuthResponseDto.java  
  entity/   → Utilisateur.java, RoleUtilisateur.java  
  security/ → JwtService.java, UtilisateurRepository.java  
  config/   → SecurityConfig.java, JwtAuthFilter.java  
  service/  → InscrireService.java, ConnecterService.java, ValiderCompteService.java  
  controller/ → AuthController.java, ActorController.java  
src/test/java/com/painting/auth/  
  AuthControllerTest.java, InscrireServiceTest.java, ActorControllerTest.java  
src/main/resources/ → application.properties  
pom.xml
```